UNIVERSIDADE KIMPA VITA

INSTITUTO POLITÉCNICO

CURSO DE ENGENHARIA INFORMÁTICA

Trabalho Individual de

Linguagem de Programação II

Tema: Análise de Dados de Redes Sociais (Frequência ePadrões)

Por: Manuel Joaquim Domingos

IIIº ANO

TURMA: 101

PERÍODO: MANHÃ

DOCENTES

_____

Moyo Kanivengidio

Índice

Introdução.....1
1.1. Âmbito e Justificação.....2
1.2. Objectivos Específicos.....2
2. Revisão de Conceitos Teóricos.....3
2.1. Geradores em Python.....3
2.2. Expressões Regulares.....3
2.3. Estruturas de Dados: Counter, Set e Dataclass.....4
3. Metodologia e Implementação.....5
3.1. Arquitectura do Projecto.....5
3.2. Gerador de Leitura (core/reader.py).....6
3.3. Extracção com Expressões Regulares (core/extractor.py).....6
3.4. Motor de Análise (core/analyzer.py).....7
4. Resultados da Análise.....8
4.1. Resumo Geral do Corpus.....8
4.2. Top 10 — Palavras Mais Frequentes.....9
4.3. Top 10 — Hashtags Mais Populares.....10
4.4. Top 10 — Menções Mais Frequentes.....11
4.5. URLs Identificados.....11
5. Cumprimento dos Critérios de Avaliação.....13
Conclusão.....15
Referências bibliográficas.....16

Introdução

O presente trabalho insere-se no âmbito da disciplina de Linguagem de Programação (LP), do curso de Informática do 3.º ano da Universidade Kimpa Vita — Instituto Politécnico de Uíge. O trabalho tem como tema central a análise computacional de dados provenientes de redes sociais, com particular enfoque na identificação de padrões de frequência em publicações textuais simuladas.

As redes sociais constituem hoje uma das principais fontes de dados não estruturados disponíveis em larga escala. A capacidade de processar, filtrar e extrair padrões a partir desses dados representa uma competência técnica de elevado valor no mercado de trabalho contemporâneo, particularmente no contexto angolano, onde a transformação digital tem vindo a acelerar de forma notável nos últimos anos.

1.1. Âmbito e Justificação

Este trabalho prático permite consolidar, de forma aplicada, um conjunto de conceitos fundamentais da linguagem Python abordados ao longo do semestre: geradores, expressões regulares, estruturas de dados nativas e modularidade do código. A escolha da temática de redes sociais não é arbitrária — reflecte a relevância crescente da análise de texto e do processamento de linguagem natural (NLP) como áreas de aplicação directa dos conhecimentos adquiridos.

Conforme enunciado pelo docente Prof. Moyo Kanivengidio na sebenta de apoio "Linguagem de Programação Python Avançado", a eficiência no processamento de dados e a correcta utilização de estruturas de dados são pilares fundamentais para o desenvolvimento de aplicações Python de qualidade profissional.

1.2. Objectivos Específicos

Para além do objectivo geral de analisar frequências num corpus de posts simulados, o trabalho definiu os seguintes objectivos específicos:

- Implementar um gerador Python para leitura eficiente de ficheiros de texto de grande dimensão;
- Aplicar expressões regulares para extracção precisa de entidades (hashtags, menções, URLs);
- Utilizar adequadamente as estruturas Counter, frozenset e dataclass do Python;
- Construir uma interface de utilizador interactiva no terminal com menu de navegação;
- Apresentar resultados de forma clara e interpretável, com análise crítica dos padrões identificados.

2. Revisão de Conceitos Teóricos

Esta secção sintetiza os principais conceitos teóricos que fundamentam as escolhas de implementação adoptadas no trabalho, estabelecendo a ponte entre a teoria abordada nas aulas e a prática desenvolvida.

2.1. Geradores em Python

Um gerador em Python é uma função que utiliza a instrução yield em vez de return. Quando chamada, não executa imediatamente o corpo da função, mas retorna um objecto iterador. A execução é retomada a cada chamada a next(), prosseguindo até ao próximo yield ou até ao fim da função.

A principal vantagem dos geradores reside na eficiência de memória: ao contrário de uma lista, que carrega todos os elementos em memória simultaneamente, um gerador produz cada elemento apenas quando necessário (avaliação lazy). Para o processamento de ficheiros de texto com potencialmente milhares de linhas, esta propriedade é crucial.

Conforme descrito na sebenta de apoio do docente, os geradores são um mecanismo avançado de Python que permite escrever código mais eficiente e expressivo, seguindo o princípio da separação entre a lógica de produção e de consumo de dados.

```
def ler_posts(caminho: str):
    with open(caminho, "r", encoding="utf-8") as f:
        for linha in f:
            linha = linha.strip().lower()
            if linha:
               yield linha  #produz uma linha de cada vez
```

Figura 1 — Implementação do gerador de leitura (core/reader.py)

2.2. Expressões Regulares

Expressões regulares (regex) são sequências de caracteres que definem padrões de pesquisa em strings. O módulo **`re`** da biblioteca padrão do Python disponibiliza funções como **`re.compile(), re.findall() e re.sub()`** para trabalhar com esses padrões.

A compilação prévia dos padrões com **`re.compile()`** é considerada boa prática quando um mesmo padrão será aplicado múltiplas vezes, pois o padrão compilado é armazenado em cache, reduzindo o custo computacional de recompilação. No presente trabalho, foram compilados quatro padrões distintos para cobrir os tipos de entidades relevantes: hashtags, menções, URLs e limpeza de pontuação.

A utilização da flag `re.IGNORECASE` garante que os padrões funcionam independentemente da capitalização do texto, o que é particularmente relevante após a conversão para minúsculas na fase de pré-processamento.

2.3. Estruturas de Dados: Counter, Set e Dataclass

O módulo **collections** disponibiliza a classe Counter, uma subclasse de dict especializada na contagem de elementos. O método update() permite acumular contagens de forma incremental, e o método most_common(n) retorna os n elementos mais frequentes de forma eficiente.

O tipo frozenset foi utilizado para armazenar as stopwords, dado que é imutável e oferece verificação de pertença em tempo O(1) — significativamente mais eficiente do que uma lista, onde a verificação é O(n). Para colecções de lookup fixas como as stopwords, o frozenset é a estrutura de dados óptima.

A dataclass AnalisadorResultado foi implementada com o decorador @dataclass, que gera automaticamente os métodos __init__, __repr__ e outros. A adição de propriedades calculadas (@property) permite encapsular estatísticas derivadas sem as calcular desnecessariamente.

3. Metodologia e Implementação

A solução foi desenvolvida em Python 3.12, adoptando uma arquitectura modular inspirada nos princípios do design orientado a responsabilidades únicas (Single Responsibility Principle). O projecto está organizado em dois pacotes principais e um directório de dados.

3.1. Arquitectura do Projecto

Módulo	Ficheiro	Responsabilidade
core	**stopwords.py**	Definição da lista de stopwords em português (frozenset)
core	**reader.py**	Gerador de leitura e pré-processamento linha por linha
core	**extractor.py**	Extracção de entidades via regex (hashtags, menções, URLs, palavras)
core	**analyzer.py**	Motor central: AnalisadorResultado, Counter, pesquisa, estatísticas
ui	**terminal.py**	Utilitários de terminal: cores ANSI, limpeza de ecrã inteligente
ui	**report.py**	Renderização de todos os relatórios formatados
ui	**menu.py**	Menu interactivo com 8 opções, validação de input e loop principal
data	**posts.txt**	Corpus de 85 posts simulados de redes sociais angolanas
—	**main.py**	Ponto de entrada: inicializa o sys.path e lança o menu

Tabela 1 — Arquitectura modular do projecto social_analyzer

3.2. Gerador de Leitura (core/reader.py)

A função ler_posts() foi implementada como um gerador Python, garantindo eficiência de memória O(1) independentemente do tamanho do ficheiro. O pré-processamento executado em cada linha compreende três etapas sequenciais:

- Remoção de espaços em branco nas extremidades (str.strip());
- Conversão para minúsculas (str.lower()) — normalização necessária para comparação de entidades;
- Remoção de pontuação básica via str.translate(), preservando os caracteres # @ / : . indispensáveis para a extracção de entidades.

O módulo inclui ainda a função auxiliar contar_linhas(), que reutiliza o gerador para contar o número de posts sem carregar o ficheiro em memória.

3.3. Extracção com Expressões Regulares (core/extractor.py)

Quatro padrões regex foram compilados estaticamente ao nível do módulo, sendo partilhados por todas as chamadas às funções de extracção:

Função	Padrão Regex	Entidade Extraída
extrair_hashtags()	**#([a-z0-9_áéíóú]+)**	Texto após # (ex: python de #python)
extrair_mencoes()	**@([a-z0-9_áéíóú]+)**	Nome de utilizador após @ (ex: joao de @joao)
extrair_urls()	**https?://[^\s]+**	URL completo (http ou https)
extrair_palavras()	**tokenização + filtro**	Palavras com ≥3 chars, alfabéticas, sem stopwords

Tabela 2 — Padrões regex e respectivas funções de extracção

A função extrair_palavras() aplica um processo de três etapas antes de tokenizar: remove URLs, hashtags e menções do texto, e depois filtra tokens com menos de 3 caracteres, tokens não-alfabéticos e palavras presentes no frozenset de stopwords.

3.4. Motor de Análise (core/analyzer.py)

A função analisar_ficheiro() constitui o núcleo da aplicação. Itera sobre o gerador, processa cada post e acumula resultados numa instância de AnalisadorResultado — uma dataclass que encapsula todos os contadores e estruturas de dados:

```
resultado.contador_palavras.update(palavras)
# Counter acumula
resultado.contador_hashtags.update(hashtags)
# Counter acumula
resultado.contador_mencoes.update(mencoes)
# Counter acumula
resultado.todas_urls.update(urls)
# set: apenas únicos
```

Figura 2 — Acumulação de contagens no motor de análise

A função obter_estatisticas_avancadas() calcula métricas derivadas: total de ocorrências por categoria, média de hashtags por post e índice de diversidade do vocabulário (razão entre palavras únicas e total de ocorrências, expressa em percentagem).

A função pesquisar_posts() implementa uma pesquisa linear sobre o corpus, suportando pesquisa por palavra comum, por hashtag (com prefixo #) ou por menção (com prefixo @), e retornando os pares (número_do_post, texto) de todos os posts que contêm o termo pesquisado.

3.5. Interface de Utilizador

O pacote ui/ foi desenvolvido em três camadas distintas. O módulo terminal.py centraliza toda a lógica de formatação visual: detecção de suporte a sequências ANSI, definição de constantes de cor, limpeza de ecrã inteligente (com fallback para sistemas sem suporte ANSI), e funções auxiliares para input validado e pausa.

O módulo report.py contém exclusivamente funções de renderização, sem qualquer lógica de negócio. Esta separação permite que os relatórios sejam reutilizados em contextos diferentes (terminal, ficheiro, etc.) sem alterações.

O módulo menu.py implementa o loop principal da aplicação, com 8 opções de navegação, animação de loading durante o carregamento dos dados, e gestão de erros (Ctrl+C é tratado graciosamente sem traceback).

4. Resultados da Análise

O corpus analisado é composto por 85 posts simulados, criados especificamente para este trabalho com o intuito de reflectir o perfil temático da comunidade tecnológica angolana. Os posts foram redigidos em português e incluem terminologia técnica de programação, referências geográficas e culturais angolanas, e entidades das três categorias alvo (hashtags, menções e URLs).

4.1. Resumo Geral do Corpus

Total de posts analisados	**85**
Total de palavras únicas (vocabulário)	253
Total de ocorrências de palavras (corpus)	439
Índice de diversidade do vocabulário	57,6%
Total de hashtags únicas	92
Total de ocorrências de hashtags	265
Média de hashtags por post	3,12
Hashtag mais frequente	**#python — 37 ocorrências**
Total de menções únicas	30
Total de ocorrências de menções	31
Menção mais frequente	@analuisadev — 2 ocorrências
Total de URLs encontrados (com repetição)	26
Total de URLs únicos	26

Tabela 3 — Resumo geral das métricas do corpus analisado

O índice de diversidade do vocabulário de 57,6% indica que mais de metade das palavras utilizadas no corpus são únicas, o que reflecte uma variedade temática considerável. Este valor é coerente com um corpus de curta extensão por post (característica das redes sociais).

4.2. Top 10 — Palavras Mais Frequentes

A tabela seguinte apresenta as dez palavras com maior frequência de ocorrência no corpus, após exclusão de stopwords. A dominância de "python" (25 ocorrências) e "dados" (17 ocorrências) reflecte a orientação tecnológica dos posts simulados.

Pos.	Palavra	Ocorrências	Distribuição relativa
1	**python**	25	25
2	**dados**	17	17
3	**hoje**	16	16
4	**angola**	14	14
5	**análise**	13	13
6	**incrível**	5	5
7	**aprendendo**	5	5
8	**partilhou**	5	5
9	**forma**	5	5
10	**comunidade**	4	4

Tabela 4 — Top 10 palavras mais frequentes (excluindo stopwords)

É digno de nota que "angola" ocupa a 4.ª posição (14 ocorrências), evidenciando a forte contextualização geográfica dos posts. A presença de "análise" em 5.º lugar (13 ocorrências) é um artefacto da temática do próprio trabalho, que gerou posts sobre análise de dados.

4.3. Top 10 — Hashtags Mais Populares

As hashtags constituem a forma mais explícita de categorização de conteúdo nas redes sociais. A análise das hashtags permite identificar os tópicos de interesse da comunidade simulada de forma mais directa do que as palavras comuns.

Pos.	Hashtag	Ocorrências	Distribuição relativa
1	**#python**	37	37
2	**#angola**	28	28
3	**#tecnologia**	19	19
4	**#programação**	14	14
5	**#ia**	9	9
6	**#dados**	9	9
7	**#inovação**	7	7
8	**#datascience**	6	6
9	**#machinelearning**	6	6
10	**#futuro**	5	5

Tabela 5 — Top 10 hashtags mais populares no corpus

O domínio de #python (37 ocorrências) e #angola (28 ocorrências) confirma os dois eixos temáticos centrais do corpus: a tecnologia de programação e o contexto angolano. A presença de #ia (9 ocorrências) e #machinelearning (6 ocorrências) revela uma tendência de interesse pelas áreas de inteligência artificial, coerente com o cenário global da tecnologia.

4.4. Top 10 — Menções Mais Frequentes

As menções (@utilizador) identificam actores da rede social simulada. A distribuição das menções no corpus é notavelmente uniforme: com excepção de @analuisadev (2 ocorrências), todos os utilizadores mencionados aparecem apenas uma vez.

Pos.	Menção	Ocorrências	Distribuição relativa
1	**@analuisadev**	2	2
2	**@techangola**	1	1
3	**@mariasilva**	1	1
4	**@joaotech**	1	1
5	**@conferenciatech**	1	1
6	**@anadev**	1	1
7	**@aiangola**	1	1
8	**@carloscode**	1	1
9	**@pedroml**	1	1
10	**@luandatech**	1	1

Tabela 6 — Top 10 menções mais frequentes no corpus

A distribuição quase uniforme das menções (29 dos 30 utilizadores únicos são mencionados exactamente uma vez) indica uma rede social simulada sem um influenciador dominante claro. Este padrão é característico de comunidades técnicas emergentes, onde o conhecimento e a atenção são distribuídos de forma descentralizada.

4.5. URLs Identificados

Foram identificados 26 URLs no corpus, todos únicos — não existindo repetições. A distribuição dos URLs por domínio revela três categorias principais de fontes referenciadas nos posts simulados:

- Documentação técnica oficial: python.org, docs.python.org, pandas.pydata.org, nltk.org, networkx.org;
- Plataformas de desenvolvimento: github.com (3 URLs), medium.com (2 URLs), regex101.com;

Domínios angolanos (.ao): anadev.co.ao, angolastartup.ao, cursopython.ao, luandatech.ao, pythonangola.ao, sofiavisualizacao.ao, techangola.ao, techconf2026.ao, ml-angola.co.ao, ia.angola.co.ao.

Nº	URL
1	**https://anadev.co.ao**
2	**https://angolastartup.ao/noticia**
3	**https://beautifulsoup.readthedocs.io**
4	**https://cursopython.ao**
5	**https://docs.python.org**
6	**https://fastapi.tiangolo.com**
7	**https://github.com/amadeudev**
8	**https://github.com/carloscode**
9	**https://github.com/sentiment-angola**
10	**https://ia.angola.co.ao/artigo**
11	**https://luandatech.ao**
12	**https://medium.com/@analuisadev**
13	**https://medium.com/nlp-tutorial**
14	**https://ml-angola.co.ao**

Nº	URL
15	**https://networkx.org**
16	**https://nltk.org**
17	**https://pandas.pydata.org**
18	**https://playstore.google.com/app123**
19	**https://python.org**
20	**https://python.org/downloads**
21	**https://pythonangola.ao**
22	**https://regex101.com**
23	**https://sofiavisualizacao.ao**
24	**https://spark.apache.org**
25	**https://techangola.ao/evento2026**
26	**https://techconf2026.ao**

Tabela 7 — Lista completa dos 26 URLs únicos identificados no corpus

5. Cumprimento dos Critérios de Avaliação

A tabela seguinte apresenta o mapeamento detalhado entre cada critério de avaliação definido no enunciado e a respectiva implementação no projecto.

Critério	Peso	Implementação
Uso de Geradores	**15%**	reader.py — função ler_posts() com yield; processa o ficheiro linha por linha em memória O(1); reutilizado

Critério	Peso	Implementação
		em contar_linhas()
Manipulação de Strings e Regex	**30%**	extractor.py — 4 padrões compilados com re.compile(); pré-processamento em 3 etapas (strip, lower, translate); extracção correcta de hashtags, menções, URLs e palavras
Estruturas de Dados	**25%**	Counter (contagens), frozenset (stopwords O(1)), set (URLs únicos), list (tokens), dataclass com @property (AnalisadorResultado)
Análise de Frequência	**20%**	Counter.update() em loop; most_common(n) para rankings; estatísticas avançadas: diversidade (57,6%), média hashtags/post (3,12), hashtag dominante
Organização e Legibilidade	**10%**	9 módulos com responsabilidade única; docstrings em todas as funções; typehints; comentários inline; separação core/ui; nomenclatura em português

Tabela 8 — Mapeamento entre critérios de avaliação e implementação

Para além dos requisitos mínimos do enunciado, foram implementadas melhorias que demonstram domínio avançado da linguagem e preocupação com a qualidade da solução:

- Menu interactivo com 8 opções, loop de navegação e validação de input com feedback ao utilizador;
- Limpeza de ecrã inteligente com detecção automática de suporte a sequências ANSI e fallback para sistemas sem suporte;
- Pesquisa em tempo real no corpus por palavra, hashtag ou menção, com destaque do termo nos resultados;
- Estatísticas avançadas: índice de diversidade do vocabulário e média de hashtags por post;
- Uso de dataclass com propriedades calculadas (@property) para encapsular estatísticas derivadas;

Animação de loading durante o carregamento do ficheiro para melhor experiência de utilizador.

Conclusão

O trabalho prático foi concluído com sucesso, cumprindo na íntegra todos os requisitos definidos no enunciado e introduzindo melhorias adicionais que enriquecem a solução tanto tecnicamente como em termos de experiência de utilização. A análise do corpus de 85 posts permitiu identificar padrões claros e tirar conclusões significativas sobre o conteúdo simulado.

Do ponto de vista técnico, o trabalho consolidou competências em quatro áreas fundamentais da programação Python avançada: a utilização eficiente de geradores para processamento de dados em fluxo; a aplicação de expressões regulares para extracção de padrões em texto não estruturado; a escolha criteriosa de estruturas de dados (Counter, frozenset, dataclass) em função das características do problema; e a organização modular do código para garantir manutenibilidade e legibilidade.

A realização deste trabalho confirmou a relevância da análise de texto como área de aplicação directa dos conteúdos programáticos da disciplina, e demonstrou que Python oferece os mecanismos necessários para construir ferramentas de análise eficientes e bem estruturadas sem recurso a bibliotecas externas.

Como direcções para trabalho futuro, identificam-se: a integração com APIs reais de redes sociais (Twitter/X, Instagram); a adição de visualizações gráficas com matplotlib ou seaborn; a implementação de técnicas de stemming/lematização para análise semântica; e o desenvolvimento de uma interface web com Flask ou FastAPI.

Referências bibliográficas

- **Kanivengidio, M.** (2025). *Linguagem de Programação Python Avançado: Sebenta para o Ensino Superior*. Universidade Kimpa Vita — Instituto Politécnico de Uíge. Material de apoio pedagógico não publicado.
- OpenAI. (2026). ChatGPT (modelo GPT-5.3). Disponível em: https://openai.com/chatgpt